



Solidity智能合约开发

从零到精通

第3.3课：函数与修饰符

```
function myFunction( params ) public view
```

讲师：[Layer]



今天的学习内容



函数基本结构

函数的完整结构与组成部分



函数可见性修饰符

public、external、internal、private



状态修饰符

view、pure、payable



自定义Modifier

权限控制与状态检查



函数重载

同名函数不同参数



最佳实践

函数设计与常见错误

函数基本结构

```
function setName ( string memory _name ) public nonReentrant onlyOwner returns ( bool ) {  
  
    // 函数体  
  
}
```

🔑 必须部分

- `function` 关键字
- 函数名
- 参数列表
- 可见性修饰符

可选部分

- 参数列表（可为空）
- 状态修饰符（view/pure/payable）
- 自定义修饰符
- `returns`（返回值类型）

函数体

包含具体逻辑的代码块，定义函数的行为

```
// 示例:  
name = _name;  
emit NameChanged(_name);
```

💡 示例

```
function setValue(uint _value) public view onlyOwner returns (bool) { return true; }
```

函数组成部分

```
function 函数名(参数列表) 可见性修饰符 状态修饰符 自定义修饰符 returns (返回类型) { // 函数体 }
```

必须部分：



function 关键字

用于声明一个函数



函数名

标识函数的唯一名称



可见性修饰符

定义函数可以被哪些实体调用

可选部分：



参数列表

函数接受的输入值，可以有零个或多个参数



状态修饰符

如view、pure、payable，声明函数对状态的影响



自定义修饰符

通过modifier关键字定义，用于在函数执行前添加检查



returns (返回值)

声明函数执行后返回的数据类型

可见性修饰符概览

public



任何人可以调用

- ✓ 外部可以调用
- ✓ 内部可以调用
- ✓ 继承合约可以调用

external



只能从外部调用

- ✓ 外部可以调用
- ✗ 内部不能直接调用
- ✓ 继承合约可以调用

internal



只能内部和继承合约调用

- ✗ 外部不能调用
- ✓ 内部可以调用
- ✓ 继承合约可以调用

private



只能本合约内部调用

- ✗ 外部不能调用
- ✓ 内部可以调用
- ✗ 继承合约不能调用

Public: 公开函数

★ 特点



外部可以调用

任何外部账户或合约都可以通过交易或消息调用来执行public函数



内部可以调用

合约内部的其他函数可以直接调用public函数

继承合约可以调用

继承该合约的子合约也可以调用public函数

≡ 使用场景

- ✔ 对外提供的接口：作为合约的核心功能，供用户或DApp交互
- ✔ 用户可以调用的功能：例如，转账、存款、查询余额等
- ✔ 最常用的可见性：在不确定具体可见性时，public是最常见的选择

</> 代码示例

```
function publicFunction() public pure returns (stringmemory) {  
    return "This is public";>  
}
```



External: 外部函数

特点

- ✅ 外部可以调用：任何外部账户或合约都可以调用
- ❌ 内部不能直接调用：合约内部不能直接调用，需通过`this`
- ✅ 继承合约可以调用：子合约可以通过外部调用方式访问

优势

💰 比public更节省Gas

直接从calldata读取数据，避免内存复制操作

何时使用

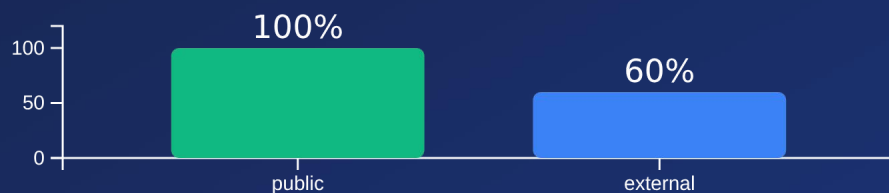
- 🔌 只给外部调用的函数
- 🔌 参数是大数组或字符串
- 🔌 追求Gas优化

</> 示例

```
function external  
Function(uint[] calldata  
data)  
external pure returns  
(uint)  
{  
    return data.length;  
}
```

💡 external直接从calldata读取，不复制到内存

Gas消耗对比



Internal: 内部函数

特点

 **外部不能调用**
外部账户或合约无法直接调用internal函数

 **内部可以调用**
合约内部的其他函数可以直接调用internal函数

 **继承合约可以调用**
继承该合约的子合约也可以调用internal函数

使用场景

 **内部辅助函数**
实现合约内部的逻辑复用，不希望对外暴露

 **可复用的逻辑**
将多个函数共享的逻辑封装成internal函数

 **给子合约继承使用**
为子合约提供可重写或直接使用的基础功能

示例

```
function internalFunction() internal pure returns (string memory) {  
    // 只能在合约内部或继承合约中调用  
    return "This is internal";  
}  
  
function testFunction() public pure returns (string memory) {  
    // 合约内部可以调用  
    return internalFunction();  
}
```


Private: 私有函数

特点

-  外部不能调用
-  内部可以调用
-  继承合约不能调用

使用场景

- > 纯内部逻辑
- > 不希望子合约访问
- > 最严格的访问控制

示例

```
function privateFunction() private pure
returns (string memory)
{
    return "This is private";
}
```

重要提示

private 变量和函数在Solidity中提供了访问控制，但并不意味着"隐私"！
区块链上所有数据都是公开的，任何人都可以读取合约的存储状态。
private仅限制了合约代码层面的访问权限，而非数据本身的保密性。

可见性修饰符对比

对比项	Public 公开	External 外部	Internal 内部	Private 私有
外部调用	✓	✓	✗	✗
内部调用	✓	✗	✓	✓
继承调用	✓	✓	✓	✗
Gas成本	中等	较低	—	—
适用场景	对外接口	外部专用	内部逻辑	私有逻辑

记忆口诀：

public 最开放， external 外部强
internal 内部用， private 最保守

如何选择可见性？

👥 外部用户需要调用？

☰ 参数有大数组？

👤 子合约需要访问？

🌐 external (省gas)

🔒 public

🏠 internal

🔒 private

最佳实践：



默认使用 internal



对外接口用 external 或 public



纯内部逻辑用 private



考虑继承用 internal

状态修饰符概览

函数与合约状态交互的方式

view (只读)

可以读取状态变量，但不能修改它们

- ✓ 可以读取状态变量
- ✓ 可以读取全局变量 (block、msg等)
- ✗ 不能修改状态变量
- ✗ 不能发送以太币

pure (纯函数)

既不读取也不修改状态变量

- ✗ 不能读取状态变量
- ✗ 不能读取msg.value、block等全局变量
- ✓ 只能使用函数参数
- ✓ 只能使用局部变量

payable (可支付)

可以接收以太币

- ✓ 可以接收以太币
- ✓ 可以读取msg.value (发送的ETH数量)
- ✓ 可以修改状态变量
- ✗ 没有payable修饰符，发送ETH会导致交易失败

View: 只读函数

view修饰符用于声明函数可以读取合约的状态变量，但不能修改它们。这类函数通常用于查询合约的当前状态。

✓ 特点

- ✓ 可以读取状态变量
- ✓ 可以读取全局变量，如block和msg等
- ✗ 不能修改状态变量
- ✗ 不能发送以太币

🛢 Gas消耗

- 当从合约外部直接调用view函数时，不消耗Gas
- 当从合约内部调用view函数时，会消耗Gas

</> 代码示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Example {
    uint256 counter = 0;

    function getCounter() public view returns (uint256) {
        return counter; // 读取状态变量
    }

    function calculate() public view returns (uint256) {
        return counter * 2; // 读取并计算
    }
}
```

💡 类比

view函数就像一个数学函数 $f(x) = x + 1$ ，给定相同的输入，总是产生相同的输出，不依赖任何外部状态。

Pure: 纯函数

✓ 特点

- ✗ 不能读取状态变量
- ✗ 不能读取 msg.value、block 等全局变量
- ✓ 只能使用函数参数
- ✓ 只能使用局部变量

⚡ Gas消耗

- 外部直接调用：不消耗Gas
- 内部调用：消耗Gas

💡 类比

pure函数就像一个数学函数：

$$f(x) = x + 1$$

给定相同的输入，总是产生相同的输出，不依赖任何外部状态

</> 代码示例

```
function add(uint a, uint b) public pure returns (uint) {  
    return a + b; // 只使用参数进行计算  
}
```

★ 最佳实践

- 能用pure就用pure
- 用于表示纯数学计算或逻辑
- 提高代码可读性和可预测性

Payable: 可支付函数

✓ 特点

- 可以接收以太币 (ETH)
- 可以读取 `msg.value` (表示随调用发送的ETH数量)
- 可以修改状态变量

⚠ 没有payable修饰符的函数

- ✗ 如果调用时发送ETH, 交易会被拒绝
- ✗ 尝试访问 `msg.value` 会报错

💡 使用场景

当函数需要处理用户付款、接收手续费或实现其他需要ETH交互的功能时, 必须使用payable修饰符。

</> 可支付函数示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract DepositContract {
    uint256 public balance;

    // 可以接收ETH
    function deposit() public payable {
        require(msg.value > 0, "Send some ETH");
        // msg.value 是发送的ETH数量 (以wei为单位)
        balance += msg.value;
    }
}
```

</> 非可支付函数示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract NormalContract {
    function normalFunction() public {
        // 如果调用此函数时发送ETH, 交易会失败
    }

    function getBalance() public view returns (uint256) {
        // 尝试访问msg.value会报错
        return address(this).balance;
    }
}
```

状态修饰符对比

修饰符	📖 读取状态	✎ 修改状态	💰 接收ETH
默认	✓	✓	✗
view	✓	✗	✗
pure	✗	✗	✗
payable	✓	✓	✓

💡 选择建议



需要修改状态? → 默认或payable



只读取状态? → view



不读不写状态? → pure



需要接收ETH? → payable

★ **最佳实践:** 能用pure就用pure, 能用view就用view, 提高合约可读性和效率

自定义Modifier

🔍 什么是 Modifier?

函数执行前的检查点，用于在函数执行前插入特定的逻辑，例如权限控制和状态检查。

★ 重要性

- 提高代码可读性
- 增强代码可维护性
- 避免重复编写相同的检查逻辑
- 实现权限控制和状态检查

</> 语法

```
modifier 名称(参数) {  
    require(条件, "错误信息");  
    _; // 占位符  
}
```

📌 下划线 _ 的作用

- 表示函数体的位置
- 会被替换为实际函数体
- 确保修饰符在函数执行前完成检查

```
function setValue(uint _value) public onlyOwner {  
    value = _value;  
}
```

Modifier 执行流程



函数调用

外部或内部调用带有修饰符的函数



执行修饰符检查

执行修饰符中_之前的所有代码，进行条件判断



条件判断

条件满足则继续，不满足则回退交易



执行函数体

_占位符被替换为实际函数体，开始执行



继续执行修饰符

函数体执行完毕，继续执行 _remaining 修饰符代码

⚙️ Modifier 示例

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}

function setValue(uint256 _value) public onlyOwner {
    value = _value;
}
```

▶ 实际执行过程

```
function setValue(uint256 _value) public {
    require(msg.sender == owner, "Not owner");
    value = _value;
}
```

常用Modifier示例

1. 权限控制

```
modifier onlyOwner() {  
    require(msg.sender == owner, "Not the owner");  
    _;  
}
```

限制只有特定地址才能调用函数

2. 状态检查

```
modifier whenNotPaused() {  
    require(!paused, "Contract is paused");  
    _;  
}
```

确保合约处于特定状态才能执行操作

3. 参数验证

```
modifier validAddress(address _addr) {  
    require(_addr != address(0), "Invalid address");  
    _;  
}
```

在函数执行前验证输入参数的有效性

4. 最小值检查

```
modifier minValue(uint256 _min) {  
    require(msg.value >= _min, "Insufficient value");  
    _;  
}
```

确保交易附带的ETH达到某个最小值

 使用示例：

组合多个Modifier

```
function
criticalFunction(uint256 _value)
    public
    onlyOwner
    whenNotPaused
    validAddress(msg.sender)
{
    // 函数体
}
```

onlyOwner

权限检查

whenNotPaused

状态检查

validAddress

参数验证

函数体

实际执行

↓ 执行顺序

- 修饰符按照在函数声明中出现的**顺序从左到右**依次执行
- 只有当前一个修饰符检查通过，才会执行下一个修饰符

⚠ 回退机制

- 任何一个修饰符中的条件检查失败，交易会**立即回退**
- 函数体及后续的修饰符代码都不会被执行

函数重载

什么是函数重载？

- 在同一合约中定义多个同名函数
- 这些函数的参数列表必须不同（参数类型或数量）
- 编译器根据传入参数自动匹配正确的函数版本

⚠ 注意事项

- ✖ 只有返回值类型不同，而参数列表完全相同不能构成重载
- ℹ 函数重载在编译时确定调用哪个版本

💡 使用场景

- ✓ 同一功能不同参数灵活实现
- ✓ 提供多个版本的API以适应不同使用场景

转账函数重载示例：

```
function transfer(address to, uint amount) public {  
    // 版本1: 简单转账逻辑  
}  
  
function transfer(  
    address to,  
    uint amount,  
    string memory memo  
) public {  
    // 版本2: 带备注的转账逻辑  
}
```

调用示例：

```
transfer(addr, 100) ;           → 调用版本1  
transfer(addr, 100, "Payment") ; → 调用版本2
```

函数匹配规则



参数类型不同

function test(uint a) / function test(bytes32 a)

参数数量不同

function test(uint a) / function test(uint a, uint

可见性+状态修饰符组合

修饰符组合可以创建更精确的函数行为，以下是常见的组合及其适用场景：

public view

- ✔ 外部可调用的查询函数
- ✔ 最常用的组合
- 💡 适用：状态查询、计算属性

external pure

- ✔ 外部调用的纯计算函数
- ✔ 更省Gas
- 💡 适用：数学计算、参数验证

internal view

- ✔ 内部辅助查询函数
- ✔ 可被继承合约使用
- 💡 适用：内部逻辑复用

private pure

- ✔ 私有计算函数
- ✔ 最高访问限制
- 💡 适用：敏感计算、辅助逻辑

public payable

- ✔ 接收ETH的公开函数
- 💡 适用：存款、收款功能

💡 最佳实践：尽可能使用最严格的修饰符组合

函数设计最佳实践

可见性原则

- 默认使用最严格的可见性
- 对外接口用 `external`（大参数）或 `public`
- 内部辅助函数用 `internal`
- 纯内部逻辑用 `private`

状态修饰符原则

- 能用 `pure` 就用 `pure`
- 能用 `view` 就用 `view`
- 明确标注函数是否修改状态

Modifier原则

- 权限控制用 `modifier`
- `modifier` 名称要清晰
- 检查失败要有明确错误信息

安全原则

- `private` 不等于隐私
- 谨慎使用 `external`
- 组合 `modifier` 要考虑顺序

遵循这些原则可以提高合约的安全性、效率和可维护性

常见错误



错误1：忘记可见性

```
function test() { // 编译错误!  // 必须指定可见性 }
```

- ✖ 原因：Solidity要求所有函数必须显式指定可见性修饰符（public, external, internal, private）



错误2：internal函数external调用

```
function internal test() internal { // 外部无法调用 }
```

- ✖ 原因：internal函数只能在合约内部或继承合约中调用，不能直接从合约外部调用



错误3：view函数修改状态

```
function badView() public view { counter++; // 编译错误! }
```

- ✖ 原因：view修饰符明确表示函数不会修改合约状态



错误4：pure函数读状态

```
function badPure() public pure returns (uint) { return counter; // 编译错误! }
```

- ✖ 原因：pure修饰符明确表示函数不读取也不修改合约状态



错误5：没有payable却收ETH

```
function deposit() public { // 发送ETH会失败 }
```

- ✖ 原因：如果没有payable修饰符，但尝试接收ETH，交易将会失败并回滚

正确的函数定义

查询函数

```
function getBalance() public view returns (uint)
{
    return balance;
}
```

仅读取状态变量，不修改数据

修改函数

```
function updateBalance(uint _amount) public {
    balance = _amount;
}
```

修改合约状态变量

纯计算

```
function add(uint a, uint b) public pure returns (uint)
{
    return a + b;
}
```

不读不修改状态，仅基于参数计算

接收ETH

```
function deposit() public payable {
    require(msg.value > 0, "Send ETH");
}
```

可接收以太币，需使用payable修饰符

权限控制

```
function adminOnly() public onlyOwner {
    // 只有owner可调用
}
```

核心要点回顾



可见性修饰符关系到安全

正确选择可见性修饰符是合约安全的基础，限制函数访问范围可以降低攻击面



external比public省gas

external函数可以直接从calldata读取参数，避免内存复制，显著节省Gas消耗



private不等于数据隐私

区块链上所有数据都是公开的，private仅限制合约代码层面的访问权限



view和pure查询不消耗gas

外部直接调用view和pure函数不消耗gas，提高用户体验和效率



modifier是权限控制最佳方式

自定义modifier提高代码复用性，集中管理权限检查和状态验证



掌握函数=掌握智能合约核心

函数与修饰符是构建健壮、安全智能合约的基础，熟练掌握至关重要

知识点总结

可见性修饰符 (4种)

- ✔ **public**: 任何人可调用
- ✔ **external**: 只能外部调用, 省gas
- ✔ **internal**: 内部和继承
- ✔ **private**: 只能本合约

状态修饰符 (3种)

- ✔ **view**: 只读状态
- ✔ **pure**: 不读不写
- ✔ **payable**: 可接收ETH

自定义Modifier

- ✔ 函数执行前的检查点
- ✔ 权限控制与状态检查
- ✔ 可组合使用
- ✔ 可带参数

函数重载

- ✔ 同名函数, 参数不同
- ✔ 根据参数类型和数量匹配
- ✔ 返回值不同不能重载

今日作业

任务1：三角色权限管理合约

要求：

- 定义角色枚举 `enum Role { Owner, Admin, User }`
- 使用 mapping 存储用户角色 `mapping(address => Role) public roles;`
- 创建三个 modifier `onlyOwner, onlyAdmin, onlyUser`
- 实现不同权限的函数：

任务2：理解题

? 问题1

public 和 external 的区别是什么？
什么时候用 external 更好？

? 问题2

view 和 pure 的区别是什么？
分别在什么场景使用？

? 问题3

多个 modifier 的执行顺序是怎样的？